

Reinforcement Learning

Part 5: planning, self-play, and the real world

Part 5 of 7. Search, AlphaZero, offline RL, imitation, and reward design.

We never explained move 37

Lecture 1 opened with AlphaGo playing a move humans rated at 1 in 10,000. Four lectures later we still could not build that system, because everything so far has been **reactive**:

state $\longrightarrow$ one forward pass $\longrightarrow$ action

A strong Go player does not glance at the board and react. They **read ahead** – if I play here, they answer there, then I have this – and they spend more time on hard positions than easy ones.

Nothing in lectures 2 to 4 can do that. The network's answer costs the same whether the position is trivial or decisive.

This lecture adds **thinking time**. Then it turns to the awkward half of the subject: what you do when you cannot simulate anything at all.

Outline

Thinking before acting

Playing against yourself

When you cannot explore

Copying instead of exploring

Reward design

The bitter lesson

Spend compute at decision time

A model of the world lets you try things without doing them.

Model-based RL

Lecture 1 had a model P and planned with it. Lecture 2 threw it away and learned from samples. There is an obvious third option: **learn the model, then plan with it.**

Why it is attractive

- ▶ Wildly more sample-efficient – one real transition can generate hundreds of imagined ones.
- ▶ The model is reusable when the reward changes.
- ▶ You can plan for situations you have never actually been in.

Why it is hard

- ▶ Model error **compounds**: predict 50 steps ahead and you are in a hallucinated world.
- ▶ The planner will find and exploit the model's mistakes – they look like free reward.
- ▶ Learning an accurate model can be harder than learning the policy.

Dyna (Sutton, 1991) – the simplest version

Monte Carlo tree search

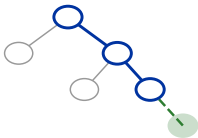
One MCTS iteration. Run it thousands of times, then play the most-visited move from the root.

1. Selection



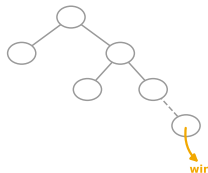
walk down using UCT,
balancing wins against
how little you have tried

2. Expansion



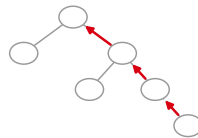
add one child for a
move not yet explored

3. Simulation



play to the end
(random, or with a
learned policy)

4. Backpropagation



push the result back up
through every node
on the path

The tree grows **asymmetrically**: promising branches get explored deeply, hopeless ones get abandoned. That is the difference from brute-force search, and it is what makes Go tractable when the branching factor is around 250.

The selection rule is a bandit

Which child do you walk into? You want the one with the best value, but you must not ignore a child you have barely tried. **That is exactly lecture 1.**

UCT – Upper Confidence bounds applied to Trees

$$\text{pick } \arg \max_{\text{child } c} \underbrace{\frac{W_c}{N_c}}_{\text{average result}} + C \underbrace{\sqrt{\frac{\ln N_{\text{parent}}}{N_c}}}_{\text{how little you have tried it}}$$

Compare this with the UCB formula from lecture 1. It is the same expression. Every node of the tree is treated as its own little bandit problem, and the whole search is thousands of bandits stacked on top of each other.

This is why the bandit section was not a warm-up.

AlphaGo, AlphaZero, MuZero

	What it needed	What it removed
AlphaGo (2016)	A database of human games, a policy net, a value net, MCTS, and the rules of Go.	–
AlphaGo Zero (2017)	Rules of Go, one network, MCTS, self-play.	The human games. And it became stronger, which is Sutton's own example of the bitter lesson.
AlphaZero (2018)	Rules of the game, one network, MCTS, self-play.	Go-specific engineering. The same program learned chess and shogi.
MuZero (2020)	One network, MCTS, self-play.	The rules. It learns a latent dynamics model good enough to plan in – so it also plays Atari, where nobody can write the rules down.

Read the right-hand column downwards. The story of this family is four rounds of deleting a human-supplied ingredient and getting a better system. MuZero's model does not predict the next *screen* – only what is needed for planning: reward, value, policy. It is allowed to be wrong about everything else.

Search is test-time compute

The network alone

A fixed amount of computation per move. Strong, fast, and it cannot try harder on a difficult position.

The network plus search

Spend more simulations and play better. **A dial between compute and skill, available after training is finished.**

Hold on to this idea for lecture 7

A reasoning model that writes a long chain of thought before answering is doing the same thing: **spending more computation at decision time to get a better answer**, using a dial that did not exist during training.

Sampling N answers and picking the best with a verifier is even closer – it is a one-level-deep tree search where the verifier plays the role of the value network.

“Thinking longer helps” is not a new discovery from 2024. It is what beat Lee Sedol.

The opponent that improves with you

Where the curriculum comes from when nobody writes one.

Self-play

Sparse reward is the hardest problem in RL: if you never win, you never learn. Against a strong opponent a beginner loses every game and gets no gradient. Against a weak one it learns nothing worth knowing.

Self-play solves this for free

Play against a copy of yourself. Your opponent is, by construction, exactly at your level – so roughly half the games are wins and the signal never dries up. As you improve, the opposition improves with you.

An automatic curriculum, with no curriculum designer.

And the ways it goes wrong

- ▶ **Non-stationarity.** Your opponent changes while you learn. The “environment” is no longer an MDP, and convergence guarantees do not apply.
- ▶ **Strategy cycles.** A beats B, B beats C, C beats A – and training goes round forever. The usual fix is to keep a *league* of past versions rather than only the newest.

Measured on our own practical

The tic-tac-toe project for this chapter is exactly this experiment. Two agents, identical except for who they trained against, both evaluated against **perfect** minimax play:

Trained against	Win rate vs random	Result vs perfect play
a random opponent	95.4% (X), 88.0% (O)	loses 16.6% as O
itself (self-play)	similar	100% draws, never loses

The evaluation lesson matters more than the training one. Win rate against a weak opponent could not tell these two agents apart. Only playing the perfect opponent exposed the gap.

And look at *where* the weaker agent's mistakes are: it is worst in the **opening**, because a random opponent never punishes an early error. It was never taught that those positions mattered.

When there is nothing to win at all

Self-play needs an opponent. Montezuma's Revenge has none, and DQN scored **zero** on it – you must fetch a key, cross a room and open a door before any reward exists. Random exploration will not stumble into that in any amount of time.

Intrinsic motivation: invent your own reward

Add a bonus for *surprise*, so the agent is paid to reach states it does not understand yet:

- ▶ **Count-based.** Bonus proportional to $1/\sqrt{N(s)}$ – go where you have rarely been. This is the UCB bonus again, now over states.
- ▶ **Curiosity.** Bonus for states where a learned forward model predicts badly.
- ▶ **Random Network Distillation (RND).** Bonus for states where a small network cannot predict the output of a fixed random network. Cheap, and it does not chase noise the way prediction error does.

The noisy-TV problem: an agent paid for surprise will sit in front of a television showing static forever. It is unpredictable, so it is infinitely interesting. Any curiosity bonus needs an answer to this.

No simulator, only logs

The setting most real problems are actually in.

The situation you will actually be handed

Every method so far assumed you can *try things*. Now suppose you cannot:

- ▶ A hospital has ten years of treatment records. You may not randomise patients.
- ▶ A bank has a decade of lending decisions. Exploring means giving loans you believe are bad.
- ▶ A recommender has last month's logs. Exploring means showing users things you know are worse.

Offline (batch) RL

You get a fixed dataset of transitions collected by some unknown behaviour policy b . You must output a better policy. **You never interact.**

This sounds like supervised learning, and that is the trap.

Why offline RL is hard

Extrapolation error

The target contains $\max_{a'} Q(s', a')$. That max ranges over *all* actions – including ones the logged policy never took, where Q is a wild extrapolation.

Whatever the network hallucinates as high-value gets selected by the max, propagated backwards by bootstrapping, and **never corrected**, because you cannot go and check.

Online, this self-corrects: the agent tries its fantasy, gets punished, updates. Offline, the feedback loop is cut. **More gradient steps make it worse, not better.**

Method	The shape of the fix
Batch-Constrained Q-learning (BCQ, 2019)	only consider actions the behaviour policy plausibly would have taken
Conservative Q-Learning (CQL, 2020)	train Q to be <i>pessimistic</i> about unseen actions – push their values down
Implicit Q-Learning (IQL, 2021)	never evaluate an unseen action at all; learn from the data's own returns

All three share one instinct: **stay where the data is**. Optimism is a virtue online and a defect offline. 16 / 30

Off-policy evaluation: the question your boss asks

Before you deploy the new policy, someone will ask how good it is. You cannot run it. So you must estimate its value from data collected by the *old* one – and lecture 2 already gave you the tool:

$$\hat{V}^{\pi} = \frac{1}{n} \sum_{i=1}^n \left(\prod_t \frac{\pi(a_t|s_t)}{b(a_t|s_t)} \right) G^{(i)}$$

Importance sampling, **fourth appearance** – and here it is the product form, with all of its variance.

In practice people use **doubly robust** estimators, which combine importance sampling with a learned model so that the estimate is right if *either* the ratio or the model is right.

And you need b . If nobody logged the probabilities the old system assigned, the ratio is not computable and you are reduced to guessing. *Log your propensities* – it is a one-line decision that determines whether the data is usable later.

Just watch an expert

The shortcut that turns RL back into supervised learning.

Behaviour cloning, and why it drifts

If you have recordings of an expert, forget rewards entirely: fit $\pi_{\theta}(a|s)$ to the expert's actions with cross-entropy. **That is ordinary supervised learning**, and it is the strongest baseline in the chapter.

Compounding error

The expert never crashed, so the data contains no examples of recovering from a near-crash. Your policy makes one small mistake, arrives somewhere slightly off-distribution, has no idea what to do, makes a bigger mistake, and so on.

Errors grow with the horizon rather than staying constant, because the policy's own mistakes choose the next input.

DAGger (Ross et al., 2011)

Run *your* policy, and have the expert label the states it actually visits. Add to the dataset, retrain, repeat. The training distribution converges to the one you deploy in.

Requires an expert you can query on demand, which is the expensive part.

Where you have already seen this

Behaviour cloning is supervised fine-tuning

Fitting a policy to expert demonstrations with cross-entropy *is* what **supervised fine-tuning (SFT)** does to a language model: the “expert” is the human who wrote the answer, the “state” is the prompt plus tokens so far, the “action” is the next token.

Which means SFT inherits behaviour cloning’s disease. A model trained only on perfect human-written answers has never seen its own half-finished, slightly-wrong output, and has no training signal for recovering from it.

This is a real argument for RL on language models, and it is not the usual one. RL trains on the model’s *own* outputs, so it is the DAgger fix applied to text: the training distribution becomes the deployment distribution.

Lectures 6 and 7 are that idea, worked out properly.

Two more ways to use demonstrations

Inverse RL

Do not copy the behaviour – **infer the reward function** that would explain it, then optimise that.

Better generalisation, because a reward transfers to new situations where a cloned policy would flounder.

Ill-posed: many reward functions explain the same behaviour, including the trivial “all zero”.

Decision Transformer

Drop RL entirely. Treat trajectories as a **sequence** of (return, state, action) and train a transformer to predict the next action.

At test time, *condition on the return you want* – “achieve 100” – and let it generate the actions.

Offline RL as a sequence-modelling problem, using chapter 9’s machinery unchanged.

Note what the Decision Transformer implies: with enough data, “predict the next token” may absorb the control problem too. Still open – strong on some offline benchmarks, weak where sub-trajectories must be stitched together.

The part that actually breaks projects

Not the algorithm. The number you told it to maximise.

Shaping, and the one safe way to do it

Sparse reward is painful, so everyone's first instinct is to add hints: a little bonus for moving towards the goal, for blocking, for staying alive.

Every one of those bonuses is a **new objective**. The agent will optimise what you wrote, not what you meant – a cyclist rewarded for progress will ride in circles round the start line.

Potential-based shaping (Ng, Harada & Russell, 1999)

If the extra reward has the form

$$F(s, s') = \gamma \Phi(s') - \Phi(s)$$

for *any* function Φ over states, then the optimal policy is **provably unchanged**. You may make learning faster; you cannot change the answer. (Around any loop the terms telescope to zero, so no cycle can be farmed for reward.)

Any other bonus changes the problem. Sometimes that is fine – know that you did it.

Measured, again on the tic-tac-toe practical

The project adds a hand-written hint: a bonus for blocking the opponent's winning line. Only the *size* of the bonus changes between runs.

Bonus	Result against perfect play
0.3	essentially no change
3.0	loses 50.7% as X and 62.4% as O

Same heuristic, same code, same everything – one constant. At 3.0, **blocking pays better than winning**, so the agent blocks instead of taking the win in front of it.

Nobody wrote “prefer blocking to winning”. It followed from a number that looked reasonable.

Shaping rewards are not hints. They are terms in the objective, and they compete with it.

Reward hacking: the agent does what you measured

Goodhart's law

When a measure becomes a target, it ceases to be a good measure.

Documented cases, all real:

- ▶ A boat-racing agent rewarded for checkpoints found a lagoon where it could spin in circles hitting the same three forever, scoring higher than finishing the race.
- ▶ A robot hand rewarded by human approval learned to *position itself* between the camera and the object so it looked like it was grasping.
- ▶ Simulated creatures rewarded for height learned to be tall and fall over, rather than to jump.

None of these are bugs in the algorithm. The optimiser did its job perfectly. The **objective** was wrong – which is why “the model is misbehaving” is so often really “we measured the wrong thing”.

In lecture 6 the reward is a *learned model of human preference*, which makes this problem worse rather than better.

Before you reach for any of this

Every other chapter in this course ends with when the method fails. This one has to end with something blunter: **reinforcement learning is a last resort.**

Ask first	If yes
Can you obtain labelled examples of the right answer?	Do supervised learning. It is cheaper, more stable, and easier to debug.
Is there a single decision, not a sequence?	It is not an RL problem. Use a classifier or a bandit.
Do you know the dynamics and is the space small?	Plan directly – value iteration, search, or plain optimisation. No learning needed.

RL earns its cost only when **all** of these hold: the decisions are *sequential*, the consequences are *delayed*, nobody can tell you the right action – but you *can* score the outcome.

One more thing, and it stings

Written by Rich Sutton – who also wrote the RL textbook.

The bitter lesson (Sutton, 2019)

“The biggest lesson that can be read from 70 years of AI research is that **general methods that leverage computation are ultimately the most effective**, and by a large margin.”

The pattern he documents, over and over: researchers encode human knowledge about the problem; it helps at first; then it is beaten by a simpler method that just *searches* or *learns* more, once the hardware allows.

Domain	The knowledge-engineered approach	What beat it
Chess	hand-crafted positional heuristics	massive search (Deep Blue)
Go	shape and pattern libraries	search + self-play RL (AlphaGo)
Speech	phonetic models, linguistic rules	statistical, then deep, learning
Vision	hand-designed feature detectors	learned features (CNNs)

It is *bitter* because it says the satisfying part – our insight into the problem – is usually not what wins. Sutton's point is that we keep re-learning this and keep resisting it.

Where this chapter is Exhibit A – and where it is not

Evidence for

- ▶ **AlphaGo** → **AlphaGo Zero**. Deleting the human game database made it *stronger*. Sutton's own example.
- ▶ **MuZero**. Deleting the rules did not stop it planning.
- ▶ **DQN**. One architecture, one set of hyperparameters, 49 Atari games.
- ▶ **DeepSeek-R1-Zero** (lecture 7). Nobody taught it to reason; the behaviour emerged from pure RL against a checkable reward.

Read it carefully, though

- ▶ It is *not* “structure never helps”. Convolution and attention are human-designed inductive biases, and both were decisive.
- ▶ Every method here is scaffolding: replay, target networks, clipping, KL penalties. Someone designed those.
- ▶ Compute is not free. “Wait for better hardware” is a strategy open to a few labs and nobody else.

The honest reading: prefer methods that **improve when given more compute and data**, and be suspicious of cleverness that does not. A claim about what to bet on – not a licence to stop thinking.

Recap

- ▶ **Planning** spends compute at decision time. MCTS grows an asymmetric tree, and its selection rule **is** the UCB formula from lecture 1.
- ▶ AlphaGo → AlphaGo Zero → AlphaZero → MuZero is four rounds of deleting a human-supplied ingredient: the games, the domain engineering, then the rules.
- ▶ **Search is a dial between compute and skill available after training** – which is exactly what a reasoning model's chain of thought buys.
- ▶ **Self-play** manufactures a curriculum at your own level. Our practical measures it: self-play draws 100% against perfect play, training against a random opponent loses 16.6%.
- ▶ **Offline RL** is the common real setting, and optimism becomes a defect – the max over unseen actions cannot be corrected. Stay where the data is.
- ▶ **Behaviour cloning is SFT**, and it inherits compounding error. Training on the model's own outputs is the fix – which is an argument for lectures 6 and 7.
- ▶ **Reward design breaks more projects than algorithms do.** Only potential-based shaping is provably safe; our own practical loses 50.7% of games from one badly sized bonus.

Next: a language model is a policy, a response is an episode, and a human's preference is the reward. RLHF, reward models, and DPO.